



Kotlin Quiz

Android Lecture

Will code run / what it will print?



```
fun printHello(name: String? = null) {  
    println("Hello $name!")  
}  
  
printHello()
```

Will code run / what it will print?



```
fun printHello(name: String? = null) {  
    println("Hello $name!")  
}  
  
printHello()
```

Hello null!

What will the code print?



```
fun printLength(name: String?) {  
    val length = name?.length ?: return  
    println("Length: $length")  
}  
  
printLength(null)
```

What will the code print?



```
fun printLength(name: String?) {  
    val length = name?.length ?: return  
    println("Length: $length")  
}  
  
printLength(null)
```

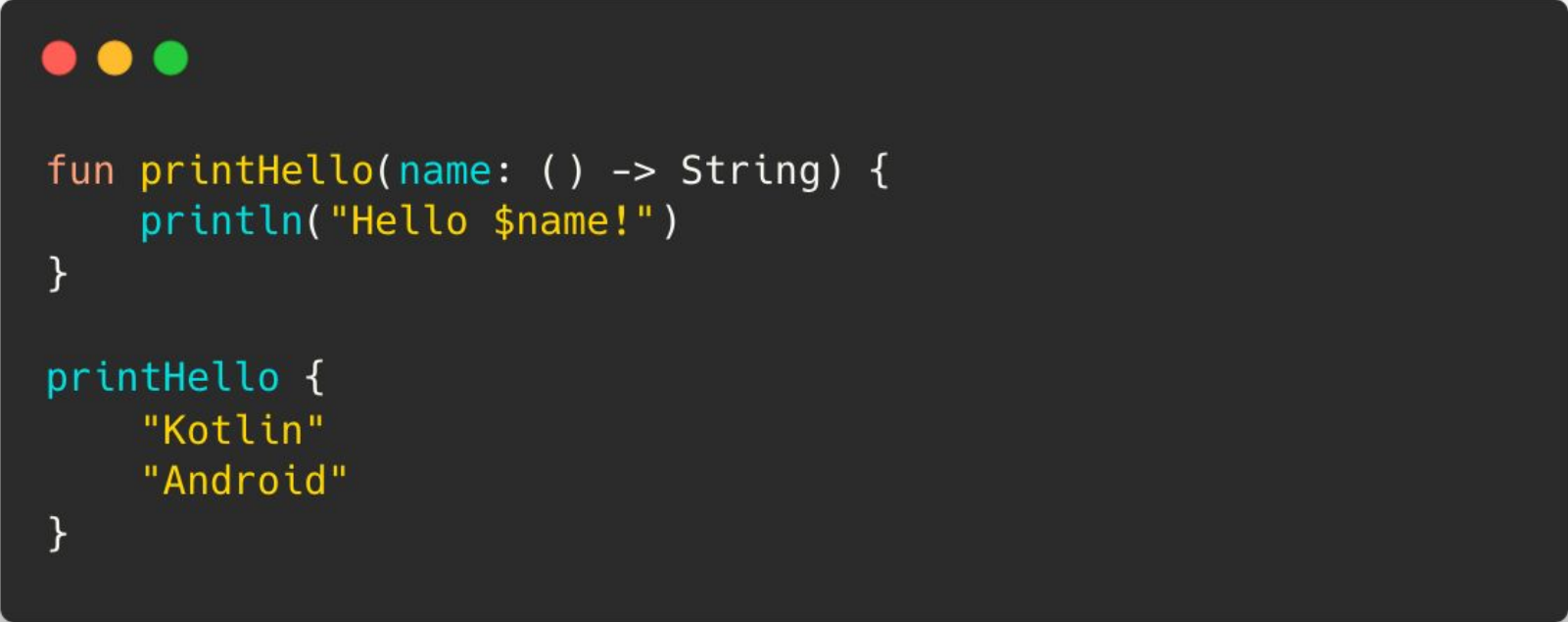
(Nothing)

Will it run / what it will print?



```
fun printHello(name: () -> String) {  
    println("Hello $name!")  
}  
  
printHello {  
    "Kotlin"  
    "Android"  
}
```

Will it run / what it will print?



```
fun printHello(name: () -> String) {  
    println("Hello $name!")  
}  
  
printHello {  
    "Kotlin"  
    "Android"  
}
```

Hello MainKt\$\$Lambda/0x0000007e01042a10@5b2133b1!

Will it run / what it will print?



```
fun printHello(name: () -> String) {  
    println("Hello ${name()}!")  
}  
  
printHello {  
    "Kotlin"  
    "Android"  
}
```

Will it run / what it will print?



```
fun printHello(name: () -> String) {  
    println("Hello ${name()}!")  
}  
  
printHello {  
    "Kotlin"  
    "Android"  
}
```

Hello Android!

Will it run / what it will print?

```
class Number(  
    var n: Int?  
) {  
    fun add(other: Int): Int =  
        if (n != null) {  
            n + other  
        } else {  
            other  
        }  
}  
  
val number = Number(3)  
println(number.add(4))
```

Will it run / what it will print?

```
class Number(  
    var n: Int?  
) {  
    fun add(other: Int): Int =  
        if (n != null) {  
            n + other  
        } else {  
            other  
        }  
}  
  
val number = Number(3)  
println(number.add(4))
```

Kotlin: Smart cast to 'Int' is impossible, because 'n' is a mutable property that could be mutated concurrently.